

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Curso de Engenharia de Software

Testes de Software

Implementando Padrões de Teste (Test Patterns)

Rafael de Paiva Gomes

Matrícula: 691992

Belo Horizonte

2026

1. Introdução

Este relatório documenta a implementação de uma suíte de testes robusta para o CheckoutService, um serviço de e-commerce que orquestra pagamento, persistência e notificação por e-mail. O objetivo é aplicar dois grupos de Padrões de Teste: os padrões de criação de dados — Object Mother e Data Builder — e os padrões de Test Doubles — Stubs e Mocks — demonstrando como cada um deles contribui para testes limpos, legíveis e sustentáveis.

2. Padrões de Criação de Dados (Builders)

2.1 Por que CarrinhoBuilder e não CarrinhoMother?

O padrão Object Mother é ideal para entidades simples e estáticas, cujos estados relevantes para os testes são poucos e bem definidos. O UserMother é um exemplo adequado: há apenas dois perfis de usuário significativos (padrão e premium), cada um representado por um método estático fixo.

O Carrinho, porém, é um objeto inerentemente composto e variável: pode conter diferentes usuários, listas de itens de tamanhos arbitrários, ou estar completamente vazio. Se utilizássemos um CarrinhoMother, seríamos forçados a criar um método para cada combinação necessária nos testes — por exemplo, umCarrinhoVazioParaUsuarioPadrao(), umCarrinhoComItemParaUsuarioPremium(), umCarrinhoComVariosItensParaUsuarioPadrao() — levando a uma explosão combinatória de métodos difíceis de descobrir e manter. Esse cenário é exatamente o Test Smell chamado "Obscure Setup", em que o leitor não consegue distinguir quais aspectos do objeto são relevantes para o teste.

O padrão **Data Builder** resolve esse problema ao fornecer uma API fluente com valores-padrão sensatos no construtor, permitindo que cada teste sobrescreva apenas o que é semanticamente relevante para aquele cenário específico.

2.2 Comparação Antes × Depois

Antes — setup manual complexo (sem Builder):

```
const user = new User('user-2', 'Usuário Premium',  
  'premium@email.com', 'PREMIUM');  
const item = new Item('Item Premium', 200);  
const carrinho = new Carrinho(user, [item]);
```

Neste setup, o leitor do teste precisa processar três linhas de construção de objetos e inferir manualmente quais atributos são determinantes para o comportamento esperado. A leitura fica prejudicada porque detalhes irrelevantes (id do usuário, nome do item) ocupam o mesmo espaço visual que detalhes essenciais (tipo PREMIUM, valor 200).

Depois — setup com Data Builder:

```
const usuarioPremium = UserMother.umUsuarioPremium();
const itens = [new Item('Item Premium', 200)];
const carrinho = new CarrinhoBuilder()
    .comUser(usuarioPremium)
    .comItens(itens)
    .build();
```

Com o Builder, a intenção do teste fica imediatamente clara: estamos criando um carrinho pertencente a um usuário premium, com um item de R\$ 200,00. Nenhum detalhe irrelevante polui a leitura. Além disso, a mudança do comportamento do Carrinho — por exemplo, adicionar suporte a cupons — exige alteração em um único lugar (o CarrinhoBuilder), e não em todos os testes que constroem carrinhos manualmente.

2.3 Benefícios para Legibilidade e Manutenção

- Intenção explícita: cada método fluente (.comUser, .comItens, .vazio) nomeia exatamente o aspecto do objeto que está sendo configurado.
- Valores-padrão seguros: o construtor do Builder inicializa o objeto com um estado válido padrão, evitando NullPointerException ou estados inválidos em testes que não precisam personalizar tudo.
- Resistência a mudanças: se a classe Carrinho ganhar um novo campo obrigatório, apenas o CarrinhoBuilder precisa ser atualizado; os testes permanecem inalterados.
- Eliminação do Obscure Setup: o leitor vê somente o que importa para aquele cenário, sem ruído de inicialização.

3. Padrões de Test Doubles (Mocks vs. Stubs)

3.1 O Teste de Referência: Cliente Premium Finaliza Compra

O teste selecionado para análise está no bloco "quando um cliente Premium finaliza a compra" do arquivo CheckoutService.test.js. Esse cenário é o mais rico do ponto de vista de Test Doubles porque envolve três dependências externas com papéis distintos: GatewayPagamento, PedidoRepository e EmailService.

3.2 Identificação dos Doubles no Teste

GatewayPagamento — usado como Stub

```
const gatewayStub = {
    cobrar: jest.fn().mockResolvedValue({ success: true })
};
```

O `GatewayPagamento` é configurado para sempre retornar `{ success: true }`, independentemente dos argumentos. Sua função primária nesse teste não é verificar se foi chamado, mas sim controlar o fluxo de execução do `CheckoutService`: garantindo que o serviço interprete o pagamento como bem-sucedido e continue o processamento. Isso é a essência de um Stub — fornecer respostas pré-programadas para controlar o caminho de execução do código sob teste.

No assert, também verificamos o argumento com o qual `cobrar` foi invocado (`expect(gatewayStub.cobrar).toHaveBeenCalledWith(180, ...)`). Esse uso secundário de verificação de argumentos é uma Verificação de Estado indireta — estamos confirmando que o cálculo de desconto produziu o valor correto, não que o `Gateway` executou algum comportamento importante em si mesmo.

PedidoRepository — usado como Stub

```
const pedidoSalvo = { id: 'pedido-123', carrinho, totalFinal: 180,
  status: 'PROCESSADO' };
const repositoryStub = {
  salvar: jest.fn().mockResolvedValue(pedidoSalvo)
};
```

O `PedidoRepository` também atua como Stub: retorna um objeto pedido pré-definido com id "pedido-123". Esse id será utilizado na verificação do e-mail, tornando-o um dado de controle do teste. Não nos interessa verificar se `salvar` foi chamado com determinados argumentos — o que importa é que o `CheckoutService` receba um pedido com id conhecido para que possamos verificar o conteúdo do e-mail.

EmailService — usado como Mock

```
const emailMock = {
  enviarEmail: jest.fn().mockResolvedValue(true)
};
```

O `EmailService` é o único double genuinamente usado como Mock neste teste. Diferentemente dos Stubs, o Mock é configurado com expectativas de comportamento que serão verificadas ao final do teste:

```
expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1);
expect(emailMock.enviarEmail).toHaveBeenCalledWith(
  'premium@email.com',
  'Seu Pedido foi Aprovado!',
  expect.stringContaining('pedido-123')
);
```

Aqui estamos fazendo Verificação de Comportamento: o teste falhará se `enviarEmail` não for chamado exatamente uma vez, com o endereço correto, o assunto correto e um corpo que contenha o id do pedido. Não nos importa o valor de retorno do e-mail — importa que a interação ocorreu corretamente.

3.3 Distinção Conceitual: Estado vs. Comportamento

Verificação de Estado (*State Verification*): examina o resultado final do objeto após a execução — "o método retornou null?", "o valor foi calculado corretamente?". Stubs são os doubles naturais para esse estilo de teste, pois apenas fornecem dados controlados para que o SUT produza o estado esperado.

Verificação de Comportamento (*Behavior Verification*): examina se certas interações ocorreram — "o método X foi chamado com os parâmetros Y?", "foi chamado exatamente N vezes?". Mocks são os doubles naturais para esse estilo, pois gravam as interações e permitem asserções sobre elas.

O envio de e-mail é um efeito colateral do checkout — não há estado retornado pelo CheckoutService que comprove que o e-mail foi enviado. A única forma de garantir que a notificação ocorreu corretamente é verificar a interação com o EmailService, o que exige um Mock. O gateway de pagamento, por outro lado, influencia o fluxo de retorno do método (null vs. pedido), tornando a Verificação de Estado suficiente — e um Stub, o double adequado.

4. Conclusão

O exercício de implementar esta suíte de testes evidenciou como a escolha deliberada de padrões influencia diretamente a sustentabilidade da base de código de testes. O Data Builder eliminou o Obscure Setup ao tornar explícitas apenas as variações semanticamente relevantes para cada cenário, enquanto o Object Mother proveu instâncias canônicas de entidades simples sem esforço adicional. Juntos, esses padrões transformam o setup de cada teste em uma leitura narrativa — "dado um carrinho de usuário premium com R\$ 200 em itens" — em lugar de uma sequência de chamadas construtoras opacas.

A distinção entre Stubs e Mocks, fundamentada na diferença entre Verificação de Estado e Verificação de Comportamento, revelou-se igualmente valiosa. Usar um Mock onde um Stub bastaria produziria um teste frágil, que quebra por qualquer mudança na implementação interna do serviço. Usar um Stub onde um Mock é necessário significaria deixar efeitos colaterais críticos (como o envio de e-mail) completamente sem cobertura. A escolha correta do double é, portanto, uma decisão de design do teste, não apenas uma preferência técnica.

Em conjunto, os Padrões de Teste estudados — Object Mother, Data Builder, Stubs e Mocks — formam um vocabulário compartilhado que permite à equipe construir testes que não apenas passam hoje, mas continuam legíveis, independentes e fáceis de manter à medida que o sistema evolui. Esse vocabulário é a principal ferramenta para prevenir Test Smells desde a concepção, em vez de corrigi-los depois.

5. Referências

- FOWLER, Martin. Mocks Aren't Stubs. Disponível em: <https://martinfowler.com/articles/mocksArentStubs.html>
- MESZAROS, Gerard. xUnit Test Patterns: Refactoring Test Code. Addison-Wesley, 2007. Disponível em: <http://xunitpatterns.com/>
- Jest Documentation — Mock Functions. Disponível em: <https://jestjs.io/docs/mock-functions>
- BECK, Kent. Test-Driven Development: By Example. Addison-Wesley, 2002.